```tsx
import React, {
  forwardRef,
  useState,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import { Loader2, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🚀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🌓 LOCAL TYPE ISOLATION GATEWAY
export interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?:
    | "brand"
    | "secondary"
    | "outline"
    | "ghost"
    | "destructive"
    | "link"
    | "premium";
  size?: "xs" | "sm" | "md" | "lg" | "xl" | "icon";
  isLoading?: boolean;
  loadingText?: string;
  leftIcon?: LucideIcon;
  rightIcon?: LucideIcon;
  iconOnly?: boolean;
  fullWidth?: boolean;
  isDisabled?: boolean;
  asChild?: boolean;
  loadingPosition?: "left" | "right" | "center";
}

const variantStyles = {
  brand:
    "bg-brand-primary text-white hover:bg-brand-hover active:bg-brand-primary
active:scale-[0.98] shadow-sm hover:shadow-md transition-[var(--transition-fast)]",
  secondary:
    "bg-secondary text-secondary-foreground hover:bg-secondary/80
active:bg-secondary active:scale-[0.98] border border-border transition-all
duration-200 ease-out",
  outline:
    "border border-input bg-transparent hover:bg-accent hover:text-accent-foreground
active:bg-accent active:scale-[0.98] transition-all duration-200 ease-out",
  ghost:
```

```
    "bg-transparent hover:bg-accent hover:text-accent-foreground active:bg-accent
active:scale-[0.98] transition-all duration-200 ease-out",
  destructive:
    "bg-destructive text-destructive-foreground hover:bg-destructive/90
active:bg-destructive active:scale-[0.98] shadow-sm hover:shadow-md transition-all
duration-200 ease-out",
  link: "text-brand-primary underline-offset-4 hover:underline text-sm font-medium
transition-colors duration-200 ease-out",
  premium:
    "bg-gradient-to-r from-brand-primary via-brand-primary/90 to-brand-primary/80
text-white hover:from-brand-hover hover:via-brand-primary/80
hover:to-brand-primary/70 active:from-brand-primary active:via-brand-primary/90
active:to-brand-primary active:scale-[0.98] shadow-lg hover:shadow-xl
active:shadow-lg transition-[var(--transition-fast)] relative overflow-hidden",
};

const sizeStyles = {
  xs: "h-7 px-2.5 text-xs gap-1",
  sm: "h-8 px-3 text-sm gap-1.5",
  md: "h-10 px-4 py-2 text-base gap-2",
  lg: "h-12 px-6 text-lg gap-2.5",
  xl: "h-14 px-8 text-xl gap-3",
  icon: "h-10 w-10 p-0",
};

const iconSizeStyles = {
  xs: "w-3 h-3",
  sm: "w-3.5 h-3.5",
  md: "w-4 h-4",
  lg: "w-5 h-5",
  xl: "w-6 h-6",
  icon: "w-5 h-5",
};

const loadingSizeStyles = {
  xs: "w-3 h-3",
  sm: "w-4 h-4",
  md: "w-5 h-5",
  lg: "w-6 h-6",
  xl: "w-7 h-7",
  icon: "w-5 h-5",
};

const focusStyles =
  "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background";

const disabledStyles =
  "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed";
```

```
const baseStyles =
  "inline-flex items-center justify-center font-sans font-medium leading-none
rounded-surface transition-all duration-200 ease-out select-none";

export const buttonVariants = variantStyles;
export const buttonSizes = sizeStyles;

export const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  (
    {
      variant = "brand",
      size = "md",
      isLoading = false,
      loadingText,
      leftIcon: LeftIcon,
      rightIcon: RightIcon,
      iconOnly = false,
      fullWidth = false,
      isDisabled = false,
      loadingPosition = "left",
      className,
      children,
      disabled,
      ...props
    },
    ref,
  ) => {
    const isActuallyDisabled = disabled || isDisabled || isLoading;
    const hasLeftIcon = Boolean(LeftIcon) && !isLoading;
    const hasRightIcon = Boolean(RightIcon) && !isLoading;
    const showLoadingSpinner =
      isLoading && (loadingPosition !== "center" || !loadingText);
    const showLoadingText = isLoading && loadingText;

    const iconSize = useMemo(() => iconSizeStyles[size], [size]);
    const loadingSize = useMemo(() => loadingSizeStyles[size], [size]);

    const baseClasses = useMemo(
      () =>
        cn(
          baseStyles,
          variantStyles[variant],
          sizeStyles[size],
          focusStyles,
          disabledStyles,
          fullWidth && "w-full",
          iconOnly && "p-0",
          className,
        ),
      [variant, size, fullWidth, iconOnly, className],
```

```
);

const [ripple, setRipple] = useState<{
  x: number;
  y: number;
  size: number;
} | null>(null);

const handleMouseDown = useCallback(
  (e: React.MouseEvent<HTMLButtonElement>) => {
    if (isActuallyDisabled) return;
    const rect = e.currentTarget.getBoundingClientRect();
    const size = Math.max(rect.width, rect.height);
    setRipple({
      x: e.clientX - rect.left - size / 2,
      y: e.clientY - rect.top - size / 2,
      size,
    });
  },
  [isActuallyDisabled],
);

const handleMouseUp = useCallback(() => {
  setTimeout(() => setRipple(null), 300);
}, []);

const handleMouseLeave = useCallback(() => {
  setRipple(null);
}, []);

useEffect(() => {
  if (isLoading) {
    setRipple(null);
  }
}, [isLoading]);

const rippleStyles = useMemo(
  () =>
    ripple
      ? {
          width: ripple.size,
          height: ripple.size,
          left: ripple.x,
          top: ripple.y,
        }
      : {},
  [ripple],
);

return (
```

```
      <button
        ref={ref}
        className={baseClasses}
        disabled={isActuallyDisabled}
        onMouseDown={handleMouseDown}
        onMouseUp={handleMouseUp}
        onMouseLeave={handleMouseLeave}
        aria-busy={isLoading}
        aria-disabled={isActuallyDisabled}
        {...props}
      >
        {showLoadingSpinner && loadingPosition === "left" && (
          <Loader2
            className={cn(loadingSize, "animate-spin", "shrink-0")}
            aria-hidden="true"
          />
        )}
        {hasLeftIcon && LeftIcon && (
          <LeftIcon className={cn(iconSize, "shrink-0")} aria-hidden="true" />
        )}
        {!isLoading && children}
        {showLoadingText && <span>{loadingText}</span>}
        {hasRightIcon && RightIcon && (
          <RightIcon className={cn(iconSize, "shrink-0")} aria-hidden="true" />
        )}
        {showLoadingSpinner && loadingPosition === "right" && (
          <Loader2
            className={cn(loadingSize, "animate-spin", "shrink-0")}
            aria-hidden="true"
          />
        )}
        {ripple && (
          <span
            className="absolute rounded-full bg-white/30 pointer-events-none
animate-[ripple_600ms_ease-out]"
            style={rippleStyles}
            aria-hidden="true"
          />
        )}
      </button>
    );
  },
);

Button.displayName = "Button";
```